Programa:	EDraw Flowchart 1.5.4	
Protección:	ASPact 2.12 + Trial 30 días	
Objetivos:	Desempakar y evitar las limitaciones	
URL descarga:	http://www.edrawsoft.com	
Dificultad:	Facilísimo	
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6	
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS]	Fecha: 18-11-05

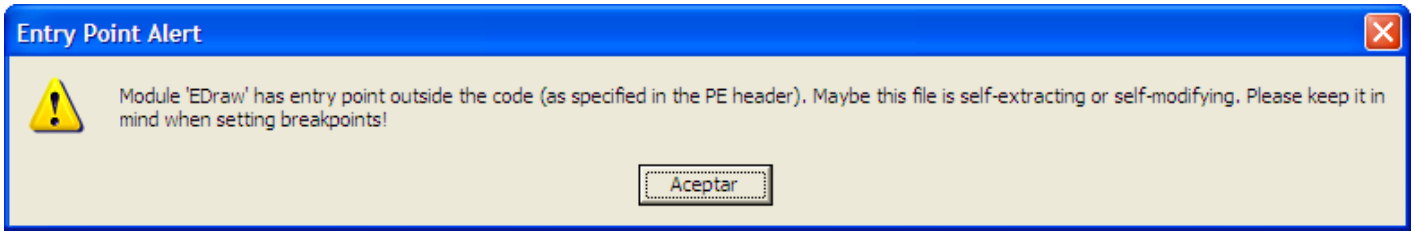
Hola a todo CracksLatinoS..!!

Una vez más estudiaremos juntos una protección, en este caso la de un programa llamado "EDraw1.5.4" del que apenas sé nada, creo que sirve para hacer dibujos cuadriculados ya que al ejecutarlo nos abre un fondo a cuadros, esto luego lo veremos, de momento vamos a abrir PEiD a ver qué nos dice:

ASPack 2.12 -> Alexey Solodovnikov

Es un ASPack 2.12 no hay que menospreciar nunca a nadie ni a nada por su puesto pero esta no es de las mejores protecciones que nos podamos encontrar, includive el autor el Sr.Solodovnikov se dio cuenta emprendiendo un nuevo camino con el ASProtect que ya es más complejo, digamos que ASPack es un ASProtect pero de un nivel netamente inferior, no tiene "stolen bytes" ni redirecciona la IAT ni nada de nada, es únicamente un compresor sencillo tipo UPX, etc…

Sabiendo esto podemos proceder al desempakado, no tardaremos ni dos minutos dada la poca dificltad, inclusive nos vale el método del PUSHAD-POPAD para hallar el OEP, pero aún más fácil es esto: vamos al EP (punto de entrada del packer)



Antes Olly nos advierte que el EP está fuera de la sección de código, este es un primer síntoma de que está comprimido (aunque ahora ya lo sabemos gracias al PEiD), aceptamos el mensaje y ahora sí par en el EntryPoint:

```
005A8001   60              PUSHAD          <-- EntryPoint..!!
005A8002   E8 03000000     CALL EDraw.005A800A
005A8007 - E9 EB045D45     JMP 45B784F7
005A800C   55              PUSH EBP
005A800D   C3              RETN
005A800E   E8 01000000     CALL EDraw.005A8014
005A8013 v EB 5D           JMP SHORT EDraw.005A8072
005A8015   BB EDFFFFFF     MOV EBX,-13
005A801A   03DD            ADD EBX,EBP
005A801C   81EB 00801A00   SUB EBX,1A8000
```

Allí quedó Olly detenido, si en este momento comenzamos a desplazarnos hacia abajo con la ruedecilla del ratón sin ejecutar código veremos esto:

```
005A838F   8B95 22040000   MOV EDX,DWORD PTR SS:[EBP+422]
005A8395 ^ E9 EBFEFFFF     JMP EDraw.005A8285
005A839A   B8 CA2F1400     MOV EAX,142FCA
005A839F   50              PUSH EAX
005A83A0   0385 22040000   ADD EAX,DWORD PTR SS:[EBP+422]
005A83A6   59              POP ECX
005A83A7   0BC9            OR ECX,ECX
005A83A9   8985 A8030000   MOV DWORD PTR SS:[EBP+3A8],EAX
005A83AF   61              POPAD           <-- El POPAD, jeje..
005A83B0 v 75 08           JNZ SHORT EDraw.005A83BA
005A83B2   B8 01000000     MOV EAX,1
005A83B7   C2 0C00         RETN 0C
005A83BA   68 00000000     PUSH 0          <-- Aquí estará el OEP!
005A83BF   C3              RETN
```

En el lugar del "PUSH 0" tras ejecutar la rutina de descompresión quedará un PUSH de la dirección del punto de entrada real del archivo (el OEP vaya), lo que hacemos es poner un BP en el POPAD y pulsar F9 para que se ejecute dicha rutina y veamos el OEP allí:

```
005A838F   8B95 22040000   MOV EDX,DWORD PTR SS:[EBP+422]
005A8395 ^ E9 EBFEFFFF     JMP EDraw.005A8285
005A839A   B8 CA2F1400     MOV EAX,142FCA
005A839F   50              PUSH EAX
005A83A0   0385 22040000   ADD EAX,DWORD PTR SS:[EBP+422]
005A83A6   59              POP ECX
005A83A7   0BC9            OR ECX,ECX
005A83A9   8985 A8030000   MOV DWORD PTR SS:[EBP+3A8],EAX
005A83AF   61              POPAD
005A83B0 v 75 08           JNZ SHORT EDraw.005A83BA
005A83B2   B8 01000000     MOV EAX,1
005A83B7   C2 0C00         RETN 0C
005A83BA   68 CA2F5400     PUSH EDraw.00542FCA   <---- OEP==542FCA
005A83BF   C3              RETN
```

Pues ya está, así de fácil obtenemos el OEP que por su puesto anotamos:

OEP: 542FCA

Este dato nos hará falta después para reparar la IAT con el Import Reconstructor, superamos el RETN y caemos en el OEP, desde aquí hacemos un DUMP que no hace falta explicar creo yo a estas alturas verdad? xDD

Tenemos ya el dumpeado que habremos llamado tute.exe para variar un poco, ahora buscamos la IAT, es fácil seguiremos un "JMP DWORD PTR [xxxxxxxx]" que sabemos que apunta a la tabla, una vez en ella subimos hasta encontrar el inicio:

```
Address   Hex dump                                          ASCII
005881BC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005881CC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005881DC  E2 68 DB 77 4D 22 DB 77 D8 17 DA 77 5B 66 DB 77  ôh■wM"■w↨rw[f■w
005881EC  9A 22 DA 77 D6 27 DA 77 BB 28 DA 77 B1 63 DB 77  ü"rwï'rw╗(rw▒c■w
005881FC  10 24 DA 77 D9 23 DA 77 00 00 00 00 00 00 00 00  ►$rw┘#rw........
0058820C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Ahí lo tenemos, tras el primer bloque encontramos muchos 00's pero no nos dejemos engañar ya que este no es el final, debemos bajar hasta ir encontrando los demás bloques, ya que cada DLL imprtada tiene el suyo propio, de momento anotamos el dato:

iat_inicio: 5881DC

Y ahora vamos a por el final, hacemos lo mismo que antes pero ahora bajando:

```
Address   Hex dump                                          ASCII
0058A208  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0058A218  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0058A228  C7 99 5C 00 6B 3E 5C 00 C6 82 5C 00 17 0E 5C 00  ╟Ö\.k>\.╞é\.↨♫\.
0058A238  C3 F3 5E 00 77 21 5C 00 EA 77 5E 00 C3 EA 5D 00  ├≤^.w!\.ûw^.├Ω].
0058A248  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0058A258  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Esos 00's si marcan el final de tabla, si miramos por debajo ya no encontramos más punteros así que ya tenemos también el final de la misma, lo anotamos también:

iat_final: 58A244

Restando estos dos valores obtenemos su tamaño:

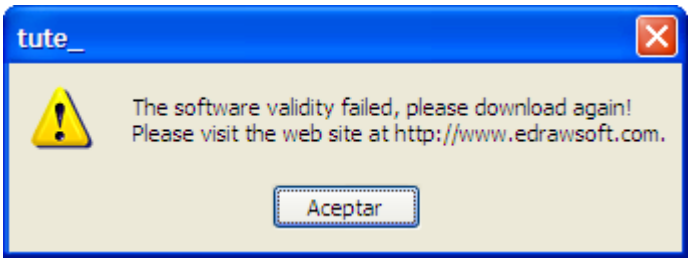
iat_largo: 2068

Una IAT un tanto grande, no es de extrañar teniendo en cuenta la cantidad de 00's que hay entre las diferentes DLL's.

Le llegó el momento al ImpRec, lo abrimos y elejimos "edraw.exe" de la list de procesos en ejecución, ponemos los datos correctos que son estos:

Pulsamos "Get Imports" y nos saca la IAT perfecta, joder qué listo es este Import no ha hecho falta hacer "Cut Thunks" reconoció los 00's y nos sacó sólo las entradas buenas al pulsar en Get Imports, ahora sólo nos queda reparar el dumpeado con un "Fix Dump" elejimos tute.exe y nos crea tute_.exe en la misma carpeta con la IAT correcta, a partir de ahora trabajaremos con este ejecutable, para obtener un serial válido no es necesario desempakar pero si tuviéramos que parchear alguna rutina mejor hacerlo antes y trabajar con un dumpeado.

Si probamos tute_.exe vemos que todavía NO funciona a la perfección, el desempakado está finalizado pero no se ejcuta bien, el desempakado ha sido muy sencillito de hacer, pero trae alguna comprobación adicional que por su puesto no forma parte de la protección sinó del propio progrma. Cuando lo ejecutamos vemos que carga bien pero una vez se abre completamente nos aparece este mensaje:



Cuando lo aceptamos nos cierra la aplicación, cargamos tute_.exe en Olly y ponemos un BP en MessageBoxA a ver si está haciendo uso de esta API para mostrar el mensaje:

Una vez puesto el BP pulsamos F9 para que cargue el programa, momentos después Olly para, el BP hizo efecto, veamos la pila:

```
0012F3F0  73D59CCB  CALL to MessageBoxA from mfc42.73D59CC5
0012F3F4  00270454  hOwner = 00270454 ('EDraw Flowchart SoftwareV1.5.4',class='Afx:400000
0012F3F8  003B7360  Text = "The software validity failed, please download again!Please
0012F3FC  003B56E8  Title = "tute_"
0012F400  00000030  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
0012F404  00000000
```

Nos dice que la llamada se realizó desde el módulo mfc42 en la dirección :73D59CC5 pero no forma parte del ejecutable esta dirección, pulsamos Ctrl+F9 para retornar, aparece el mensaje en pantalla, lo aceptamos y caemos en el RET, vamos superando RET's hasta caer en el código del .exe que hemos desempakado:

```
0049156F  .  83F9 14       CMP ECX,14
00491572  .^ 72 EA         JB SHORT tute_.0049155E
00491574  .  8A4424 17     MOV AL,BYTE PTR SS:[ESP+17]          ;ojo a [ESP+17]
00491578  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],0D
00491580  .  84C0          TEST AL,AL
00491582  .  8D4C24 18     LEA ECX,DWORD PTR SS:[ESP+18]
00491586  .v 0F85 DA00000( JNZ tute_.00491666                   ;bonito salto..
0049158C  >  E8 8D040B00   CALL <JMP.&mfc42.#800>
00491591  >  8D4C24 28     LEA ECX,DWORD PTR SS:[ESP+28]
00491595  .  889C24 B4090( MOV BYTE PTR SS:[ESP+9B4],BL
0049159C  .  E8 7D040B00   CALL <JMP.&mfc42.#800>
004915A1  .  8D4C24 10     LEA ECX,DWORD PTR SS:[ESP+10]
004915A5  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],0A
004915AD  .  E8 6C040B00   CALL <JMP.&mfc42.#800>
004915B2  .  8D8C24 90010( LEA ECX,DWORD PTR SS:[ESP+190]
004915B9  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],5
004915C1  .  E8 7F19F7FF   CALL tute_.00402F45
004915C6  >  8D4C24 1C     LEA ECX,DWORD PTR SS:[ESP+1C]
004915CA  .  E8 5B040B00   CALL <JMP.&mfc42.#540>
004915CF  .  68 4DC90000   PUSH 0C94D
004915D4  .  8D4C24 20     LEA ECX,DWORD PTR SS:[ESP+20]
004915D8  .  C68424 B8090( MOV BYTE PTR SS:[ESP+9B8],11
004915E0  .  E8 7B040B00   CALL <JMP.&mfc42.#4160>
004915E5  .  8B5424 1C     MOV EDX,DWORD PTR SS:[ESP+1C]
004915E9  .  6A 00         PUSH 0
004915EB  .  6A 00         PUSH 0
004915ED  .  52            PUSH EDX
004915EE  .  E8 AD060B00   CALL <JMP.&mfc42.#1200>              ;llama al mensaje..
004915F3  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],5          ;salimos aqui..
004915FB  .  8D4C24 1C     LEA ECX,DWORD PTR SS:[ESP+1C]
004915FF  >  E8 1A040B00   CALL <JMP.&mfc42.#800>
00491604  >  8D4C24 50     LEA ECX,DWORD PTR SS:[ESP+50]
00491608  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],2
00491610  .  E8 B1100B00   CALL <JMP.&mfc42.#617>
00491615  >  8D4C24 2C     LEA ECX,DWORD PTR SS:[ESP+2C]
00491619  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],1
00491621  .  E8 F8030B00   CALL <JMP.&mfc42.#800>
00491626  .  8D4C24 20     LEA ECX,DWORD PTR SS:[ESP+20]
0049162A  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],0
```

Aparecemos en medio de este código, la llamada al mensaje se realiza en :7915EE donde hemos puesto un BP (inútil pero bueno xDD) un poco más arriba vemos un salto tipo JNZ que de saltar evitaría el mensaje, ponemos un BP en él y reiniciamos, cuando pare invertimos el Flag_ZERO par que salte, pero ocurre que no para ¿? El motivo este:

```
004915AD  .  E8 6C040B00   CALL <JMP.&mfc42.#800>
004915B2  .  8D8C24 90010( LEA ECX,DWORD PTR SS:[ESP+190]
004915B9  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],5
004915C1  .  E8 7F19F7FF   CALL tute_.00402F45
004915C6  >  8D4C24 1C     LEA ECX,DWORD PTR SS:[ESP+1C]
004915CA  .  E8 5B040B00   CALL <JMP.&mfc42.#540>
004915CF  .  68 4DC90000   PUSH 0C94D
```

hay un salto más arriba que nos pone directos en el error, pondremos el BP allí:

```
0049136C  >  8BCD          MOV ECX,EBP
0049136E  .  E8 823EF7FF   CALL tute_.004051F5
00491373  .  85C0          TEST EAX,EAX
00491375  .v 0F84 4B02000( JE tute_.004915C6                ;este es el salto..
0049137B  .  8D8C24 90010( LEA ECX,DWORD PTR SS:[ESP+190]
00491382  .  E8 1A2EF7FF   CALL tute_.004041A1
00491387  .  8D8C24 90010( LEA ECX,DWORD PTR SS:[ESP+190]
0049138E  .  C68424 B4090( MOV BYTE PTR SS:[ESP+9B4],0A
```

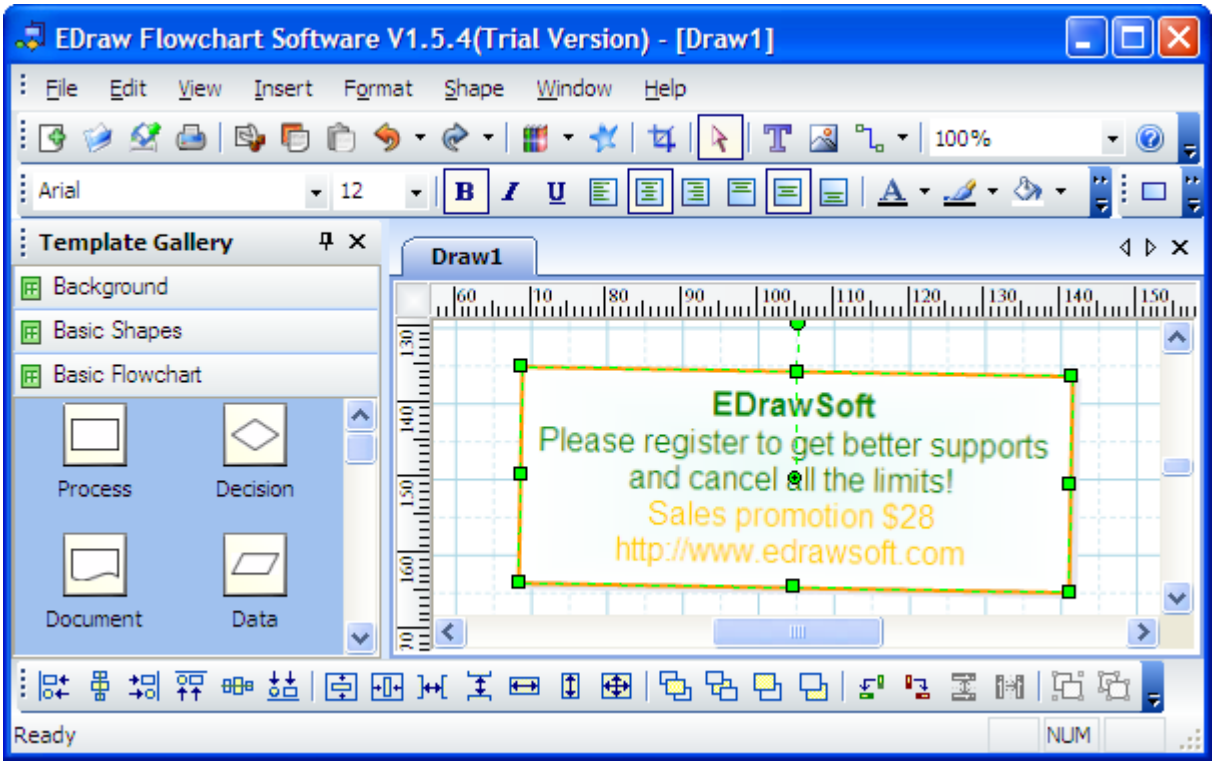
Reiniciamos Olly, pulsamos F9 y para en el CALL, entramos en él y llegamos a una zona donde va mirando ciertas cosas, al final nos saca con EAX==0 aquí:

```
00492F0C  .  89BC24 68090( MOV DWORD PTR SS:[ESP+968],EDI
00492F13  .  E8 2D00F7FF   CALL tute_.00402F45
00492F18  .  33C0          XOR EAX,EAX                      ;Pone EAX==0
00492F1A  .v EB 56         JMP SHORT tute_.00492F72         ;Y nos saca de la rutina...
00492F1C  >  68 F03C5800   PUSH tute_.00583CF0              ASCII "xiabinyes"
00492F21  .  B9 8C5A5800   MOV ECX,tute_.00585A8C
00492F26  .  E8 F9EA0A00   CALL <JMP.&mfc42.#860>
00492F2B  .  68 DC3C5800   PUSH tute_.00583CDC              ASCII "CrackdrawCrack"
00492F30  .  B9 905A5800   MOV ECX,tute_.00585A90           ASCII "PW;"
00492F35  .  E8 EAEA0A00   CALL <JMP.&mfc42.#860>
00492F3A  .  68 783B5800   PUSH tute_.00583B78              ASCII "huayesyong"
00492F3F  .  B9 885A5800   MOV ECX,tute_.00585A88
00492F44  .  E8 DBEA0A00   CALL <JMP.&mfc42.#860>
00492F49  .  8D4C24 0C     LEA ECX,DWORD PTR SS:[ESP+C]
00492F4D  .  C68424 68090( MOV BYTE PTR SS:[ESP+968],0
00492F55  .  E8 C4EA0A00   CALL <JMP.&mfc42.#800>
00492F5A  .  8D8C24 44010( LEA ECX,DWORD PTR SS:[ESP+144]
00492F61  .  89BC24 68090( MOV DWORD PTR SS:[ESP+968],EDI
00492F68  .  E8 D8FFF6FF   CALL tute_.00402F45
00492F6D  .  B8 01000000   MOV EAX,1
00492F72  >  8B8C24 60090( MOV ECX,DWORD PTR SS:[ESP+960]
```

Vemos que salta justo después del "MOV EAX,1" que es la instrucción contraria, si antes de salir del CALL cambiamos el valor de EAX por EAX==1 el programa carga bien, ya tenemos algo, de momento vamos a hacer el siguiente cambio para no tener que estar haciendo esto constantemente:

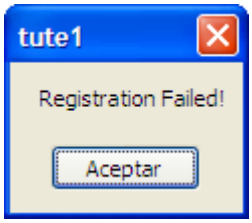
```
00492F0C  .  89BC24 68090( MOV DWORD PTR SS:[ESP+968],EDI
00492F13  .  E8 2D00F7FF   CALL tute_.00402F45
00492F18  .  33C0          XOR EAX,EAX                      ;Pone EAX==0
00492F1A  .v EB 51         JMP SHORT tute_.00492F6D         ;Vamos al "MOV EAX,1"
00492F1C  >  68 F03C5800   PUSH tute_.00583CF0              ASCII "xiabinyes"
00492F21  .  B9 8C5A5800   MOV ECX,tute_.00585A8C
00492F26  .  E8 F9EA0A00   CALL <JMP.&mfc42.#860>
00492F2B  .  68 DC3C5800   PUSH tute_.00583CDC              ASCII "CrackdrawCrack"
00492F30  .  B9 905A5800   MOV ECX,tute_.00585A90           ASCII "PW;"
00492F35  .  E8 EAEA0A00   CALL <JMP.&mfc42.#860>
00492F3A  .  68 783B5800   PUSH tute_.00583B78              ASCII "huayesyong"
00492F3F  .  B9 885A5800   MOV ECX,tute_.00585A88
00492F44  .  E8 DBEA0A00   CALL <JMP.&mfc42.#860>
00492F49  .  8D4C24 0C     LEA ECX,DWORD PTR SS:[ESP+C]
00492F4D  .  C68424 68090( MOV BYTE PTR SS:[ESP+968],0
00492F55  .  E8 C4EA0A00   CALL <JMP.&mfc42.#800>
00492F5A  .  8D8C24 44010( LEA ECX,DWORD PTR SS:[ESP+144]
00492F61  .  89BC24 68090( MOV DWORD PTR SS:[ESP+968],EDI
00492F68  .  E8 D8FFF6FF   CALL tute_.00402F45
00492F6D  .  B8 01000000   MOV EAX,1
00492F72  >  8B8C24 60090( MOV ECX,DWORD PTR SS:[ESP+960]
```

Hacemeos que el salto apunta al "MOV EAX,1" y san se acabó, salvamos los cambios como tute1.exe y a partir de ahora trabajaremos con este archivo para evitar tener que hacer esto cada vez, ahora ya carga bien xDD



Pero vemos demasiadas cosas que no nos gustan aquí, de momento nos inserta ese horrible logo pidiendo a gritos que nos registremos, por suerte se puede eliminar del archivo y no lo insereta como Watermark (marca de agua) en los documentos que creemos, además en la barra de título pone (Trial Version) que tampoco nos gusta, en fin un mogollón de pijaditas de estas, vamos a eliminarlas ;-)

Desde el menú "Help->Registration…" podemos introducir un serial y un nombre, si ponemos unos cualesquiera recibimos el siguiente mensaje de error:



Típica MessageBox, vamos al CommandBar y ponemos un BP en dicha API y volvemos a introducir el serial (en realidad no hace falta pues queda allí puesto), aceptamos y…

```
0012E8E8  73DAC8E7  CALL to MessageBoxA from mfc42.73DAC8E1
0012E8EC  001105BA  hOwner = 001105BA ('Register',class='#32770',parent=000F0164)
0012E8F0  00ED0E90  Text = "Registration Failed!"
0012E8F4  003B56E8  Title = "tute1"
0012E8F8  00000000  Style = MB_OK|MB_APPLMODAL
0012E8FC  0012F170
```

Vemos los parámetros en la pila, la llamada se hizo desde la librería mfc42.dll esto no interesa, de momento pulsamos Ctrl+F9 para llegar hasta el RET, aparecerá el mensaje en pantalla que aceptamos, superamos el RET y seguimso en la mfc de los hue… seguimos hsta el siguiente RET que está a un paso y vemos que la función devuelve el control al programa en la dirección :4D3BF8 pero aquí tampoco vemos mucho, vamos hsta el siguiente RET lo superamos y aparecemos otra vez en la mfc42, jooooder, seguimos con el método hasta llegar al código del ejecutable, esto sucede aquí:

```
00538F3D     CC            INT3
00538F3E     CC            INT3
00538F3F     CC            INT3
00538F40  >  8B4424 0C     MOV EAX,DWORD PTR SS:[ESP+C]
00538F44  .  8B5424 08     MOV EDX,DWORD PTR SS:[ESP+8]
00538F48  .  50            PUSH EAX
00538F49  .  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
00538F4D  .  52            PUSH EDX
00538F4E  .  50            PUSH EAX
00538F4F  .  E8 B48B0000   CALL <JMP.&mfc42.#2385>        ;Aqui se hace todo..
00538F54  .  C2 0C00       RETN 0C
00538F57     90            NOP
00538F58     90            NOP
00538F59     90            NOP
```

Al fin..!! puede parecer que este CALL se encarga de validar el serial, pero creo que no es así, si ponemos un BP en él vemos que para continuamente aún sin introducir ningún serial ¿? Es evidente que se lo han currado, al menos han tapado esta posibilidad bastante bien en mi opinión, pero esto no nos detendrá claro está, todavía no tengo el crack pero no nos rendiremos ;-)

He de decir que en este punto decidí cambiar de táctica, pues incluso con el wdasm seguía sin encontrarle nada, hice lo más fácil, lo que debería haber hecho de inicio: buscar las cadenas de texto, jeje.. las llamadas "String References" con Olly:

Address	Disassembly	Text string
004926CB	PUSH tute1.00583CAC	ASCII "SOFTWARE\EDrawSoft\Flowchart\UserInfo"
004926F9	PUSH tute1.00583CA8	ASCII "Num"
00492732	PUSH tute1.00583CAC	ASCII "SOFTWARE\EDrawSoft\Flowchart\UserInfo"
00492742	MOV EDI,tute1.00583CA4	ASCII "29"
00492756	PUSH tute1.00583CA4	ASCII "29"
0049275E	PUSH tute1.00583CA8	ASCII "Num"
00492DE3	PUSH tute1.00583D1C	ASCII "224H*y*@-"
00492DE8	PUSH tute1.00583D0C	ASCII "↑_wIO)[]e3d↓r"
00492E69	PUSH tute1.00583CFC	ASCII "\lookup.sp"
00492F1C	PUSH tute1.00583CF0	ASCII "xiabinyes"
00492F2B	PUSH tute1.00583CDC	ASCII "CrackdrawCrack"
00492F3A	PUSH tute1.00583B78	ASCII "huayesyong"
004930CA	PUSH tute1.00583D28	ASCII "\Config\edraw.ini"
0049325B	PUSH tute1.00583D58	ASCII "File%d"
00493260	PUSH tute1.00583D40	ASCII "Recent Files List"
00493305	PUSH tute1.00583D28	ASCII "\Config\edraw.ini"
004934EC	PUSH tute1.00583D68	ASCII "\autosaves"
0049351D	PUSH tute1.00583D60	ASCII "*.edd"
00493878	PUSH tute1.00583CAC	ASCII "SOFTWARE\EDrawSoft\Flowchart\UserInfo"
004938A6	PUSH tute1.00583D98	ASCII "Name"
004938DC	PUSH tute1.00583D90	ASCII "Serial"
00493912	PUSH tute1.00583D84	ASCII "Password"
00493AEB	PUSH tute1.00583D78	ASCII "%X-%X-%X"
00493CD0	PUSH tute1.00583CAC	ASCII "SOFTWARE\EDrawSoft\Flowchart\UserInfo"
00493CF4	PUSH tute1.00583D98	ASCII "Name"

No veas la de cosas que encontramos, pero con wdasm no las vemos ¿? Lo bueno es que desde el inicio están ahí, si lo miramos desde el OEP ya las vemos, pero bueno que más da, la cuestión es que ahí están no? Pues vamos a beneficiarnos, jeje.. ponemos un BP con F2 en cada referencia como vemos en la imagen y ahora volvamos al cuadro de registro donde pusimos nuestro nombre y el serial más falso que el judas iscariote, aceptamos y Olly para, jeje.. le hemos pillado ahora sí:

```
004D3AC9  .  51          PUSH ECX                          pDisposition
004D3ACA  .  52          PUSH EDX                          pHandle
004D3ACB  .  6A 00       PUSH 0                            pSecurity = NULL
004D3ACD  .  68 06000200 PUSH 20006                        Access = KEY_WRITE
004D3AD2  .  6A 00       PUSH 0                            Options = REG_OPTION_NON_VOLATILE
004D3AD4  .  68 D84D5800 PUSH tute1.00584DD8               Class = ""
004D3AD9  .  6A 00       PUSH 0                            Reserved = 0
004D3ADB  .  68 AC3C5800 PUSH tute1.00583CAC               Subkey = "SOFTWARE\EDrawSoft\Flowchart\UserInfo"
004D3AE0  .  68 02000080 PUSH 80000002                     hKey = HKEY_LOCAL_MACHINE
004D3AE5  .  FF15 F081580 CALL DWORD PTR DS:[<&advapi32    RegCreateKeyExA
004D3AEB  .  8B86 C803000 MOV EAX,DWORD PTR DS:[ESI+3C8
004D3AF1  .  8D8E C803000 LEA ECX,DWORD PTR DS:[ESI+3C8
004D3AF7  .  8B40 F8     MOV EAX,DWORD PTR DS:[EAX-8]
004D3AFA  .  40          INC EAX
004D3AFB  .  50          PUSH EAX
004D3AFC  .  68 04010000 PUSH 104
004D3B01  .  E8 C4E70600 CALL <JMP.&mfc42.#2915>
004D3B06  .  8B4C24 18   MOV ECX,DWORD PTR SS:[ESP+18]
004D3B0A  .  50          PUSH EAX                          Buffer
004D3B0B  .  6A 01       PUSH 1                            ValueType = REG_SZ
004D3B0D  .  6A 00       PUSH 0                            Reserved = 0
004D3B0F  .  68 983D5800 PUSH tute1.00583D98               ValueName = "Name"
004D3B14  .  51          PUSH ECX                          hKey
004D3B15  .  FF15 F881580 CALL DWORD PTR DS:[<&advapi32    RegSetValueExA
004D3B1B  .  8B5424 14   MOV EDX,DWORD PTR SS:[ESP+14]
004D3B1F  .  52          PUSH EDX                          hKey
004D3B20  .  FF15 E481580 CALL DWORD PTR DS:[<&advapi32    RegCloseKey
004D3B26  .  51          PUSH ECX
```

Ha saltado el BP de la imagen de todos los que pusimos, está creando una clave en el registro de Window$ llamada "UserInfo" después crea un valor llamado "Name" donde pondrá nuestro nombre, o sea que nada más ingresar los datos se crea su clave y se guarda nuestro nombre, poco después una llamada interesantísima:

```
004D3B45  .  8BC8        MOV ECX,EAX
004D3B47  .  E8 02100700 CALL <JMP.&cgdi.?RegeditSoft@CCrypter@@QAEHVCS   ;Ojo a este CALL..
004D3B4C  .  85C0        TEST EAX,EAX
004D3B4E  .v 74 79       JE SHORT tute1.004D3BC9
004D3B50  .  8D4C24 0C   LEA ECX,DWORD PTR SS:[ESP+C]
004D3B54  .  E8 D1DE0600 CALL <JMP.&mfc42.#540>
004D3B59  .  68 78C90000 PUSH 0C978
```

Debemos tener muy en cuenta esta llamada, en los registros no vemos nada raro pero ojo en la pila (stack) si miramos allí encontramos el serial que pusimos:

```
0012E90C  00ED0648  ASCII "1231231"
0012E910  0012F170
0012E914  00565838  tute1.00565838
0012E918  00000001
```

Ese "1231231" no es ni más ni menos que nuestro serial falso, va a entrar en un CALL cuyo parámetro que le pasa es nuestro serial ¿? Qué raro, entremos a ver qué hace con él, quizá encontremos algo de ayuda ;-) encontramos un JMP que nos envía a "CGdi.dll" que es una librería del propio programa y además está empakada con el mismo ASPack 2.12 que el .exe con el que trabajamos ¿porqué proteger tanto un .dll? buena pregunta, y más grave aún ¿Por qué se le pasa como parámetro nuestro serial? Demasiadas coincidencias como para no pensar que este CALL es importante..!! Ahora veremos…

Entramos en el CALL y caemos en CGdi.dll no pasa nada no hay problema, sólo vamos a husmear un poquitín y si hubiera que desempakar la .dll pues nada tampoco hay problema, se trata de aprender verdad? Pues ya está! Entaramos en ella y en seguida vemos cosas raras que nos llaman la atención, primero que manipula un tanto nuestro serial pues si ponemos "1231321" nos aparece como "3112123" así que de entrada le hace ya algo, no nos interesa para nada ese algo, seguimos y llegamos a esto:

```
10001475  E8 B0140000  CALL cgdi.1000292A          JMP to mfc42.#800
1000147A  8B4424 0C    MOV EAX,DWORD PTR SS:[ESP+C] ;coge el serial..
1000147E  BE 05000000  MOV ESI,5                   ;hace ESI==5..
10001483  8B40 F8      MOV EAX,DWORD PTR DS:[EAX-8] ;coge el largo del serial..
10001486  83C0 FC      ADD EAX,-4                  ;le resta 4..
10001489  3BC6         CMP EAX,ESI                 ;lo compara con 5..
1000148B  v 7E 79      JLE SHORT cgdi.10001506     ;si es inferior o igual salta..
1000148D  B3 09        MOV BL,9
1000148F  6A 01        PUSH 1
10001491  8D4C24 54    LEA ECX,DWORD PTR SS:[ESP+54]
```

Lo que hace es mirar si la longitud del serial es inferior o igual a 9, si es así tira para un lado y si no para otro, nosotros vamos con uno de 7 vamos para :10001506 de cabeza xDD

Después se mete en un bucle donde verifica si cada digito del serial sea un valor numérico (un dígito vaya), esto lo hace aquí:

```
1000168E   894424 20      MOV DWORD PTR SS:[ESP+20],EAX
10001692   894424 14      MOV DWORD PTR SS:[ESP+14],EAX
10001696 v 7E 2B          JLE SHORT cgdi.100016C3
10001698   8B4C24 20      MOV ECX,DWORD PTR SS:[ESP+20]          ;inicio_loop..
1000169C   0FBE940C 90000 MOVSX EDX,BYTE PTR SS:[ESP+ECX+90]
100016A4   52             PUSH EDX
100016A5   FF15 98910010  CALL DWORD PTR DS:[10009198]           MSVCRT.isxdigit
100016AB   83C4 04        ADD ESP,4
100016AE   85C0           TEST EAX,EAX
100016B0 v 0F84 D0000000  JE cgdi.10001786                       ;si no es un digito salta..
100016B6   8B4424 20      MOV EAX,DWORD PTR SS:[ESP+20]
100016BA   40             INC EAX
100016BB   3BC7           CMP EAX,EDI
100016BD   894424 20      MOV DWORD PTR SS:[ESP+20],EAX
100016C1 ^ 7C D5          JL SHORT cgdi.10001698                 ;fin_loop..
100016C3   85FF           TEST EDI,EDI
```

Si alguno no lo es nos envía a :10001786 donde no sé qué hará ¿? Nosotros nos quedamos porque pusimos todo dígitos, así que llegamos a :100016C3 normalmente, se mete en varios CALL con nuestro serial y cadenas larguísimas con las que parece que va a verificar algo, encriptar algo ¿? En fin que tampoco interesa, al final llegamos a una comparación que sí que interesa ;-)

```
1000178E   8D8424 A002000 LEA EAX,DWORD PTR SS:[ESP+2A0]
10001795   8D4C24 5C      LEA ECX,DWORD PTR SS:[ESP+5C]
10001799   50             PUSH EAX
1000179A   51             PUSH ECX
1000179B   FF15 24900010  CALL DWORD PTR DS:[10009024]           kernel32.lstrcmpA
100017A1   85C0           TEST EAX,EAX
100017A3   5D             POP EBP
100017A4 v 75 26          JNZ SHORT cgdi.100017CC
```

Si miramos la pila encontramos las dos cadenas que compara:

```
0012E444   0012E4A8   String1 = "SyXe'05"
0012E448   0012E6EC   String2 = "60"
0012E44C   0012E944
```

Vaya, vaya... nuestro nombre!! Y esa string2 que será? Puede ser la cadena que obtuvo haciendo operaciones con el serial, recordad que acaba de entrar en varios CALL que os comentaba antes donde parecía hacer algo de esto, en fin que si las cadenas no son iguales queda un horrible EAX==1 que nos MATA...

```
1000179B   FF15 24900010  CALL DWORD PTR DS:[10009024]           kernel32.lstrcmpA
100017A1   85C0           TEST EAX,EAX
100017A3   5D             POP EBP
100017A4 v 75 26          JNZ SHORT cgdi.100017CC
100017A6   8B4424 10      MOV EAX,DWORD PTR SS:[ESP+10]
100017AA   BB 01000000    MOV EBX,1
100017AF   3BC3           CMP EAX,EBX
100017B1 v 74 19          JE SHORT cgdi.100017CC
100017B3   85FF           TEST EDI,EDI
100017B5 v 74 15          JE SHORT cgdi.100017CC
100017B7   8B5424 3C      MOV EDX,DWORD PTR SS:[ESP+3C]
100017BB   8B7424 18      MOV ESI,DWORD PTR SS:[ESP+18]
100017BF   52             PUSH EDX
100017C0   8BCE           MOV ECX,ESI
100017C2   E8 A9070000    CALL cgdi.?WriteRegeditInfo@CCrypter@@I ;ojo a este CALL...
100017C7   895E 04        MOV DWORD PTR DS:[ESI+4],EBX
100017CA v EB 0D          JMP SHORT cgdi.100017D9
100017CC   8B4424 18      MOV EAX,DWORD PTR SS:[ESP+18]
100017D0   8BF0           MOV ESI,EAX
```

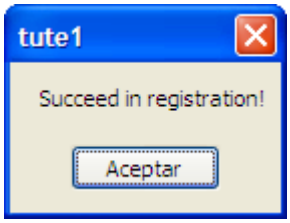
Salta en :100017A4 evitando pasar por ese CALL tan sospechoso WriteRegeditInfo@Crypter que no me gusta nada, creo que deberíamos pasar por él, pero bueno además estamos traceando un .dll como sabemos "CGdi.dll" de momento no hacemos nada, dejamos que salte y seguimos por :100017CC a ver qué más hace, se mete en algunos CALL y nos saca con el resultado EAX==0, este es el valor de retorno de la función, veamos como sigue:

```
004D3B45   .  8BC8        MOV ECX,EAX
004D3B47   .  E8 02100700 CALL <JMP.&cgdi.?RegeditSoft@CCrypte ;Ojo a este CALL..
004D3B4C   .  85C0        TEST EAX,EAX
004D3B4E   .v 74 79       JE SHORT tute1.004D3BC9
004D3B50   .  8D4C24 0C   LEA ECX,DWORD PTR SS:[ESP+C]
```

Sigue con un salto en :4D3B4E que ya está dentro del ejecutable, seguimos el salto y llegamos a la zona donde nos saca el mensaje de error:

```
004D3BEE   .  6A 00       PUSH 0
004D3BF0   .  52          PUSH EDX
004D3BF1   .  8BCE        MOV ECX,ESI
004D3BF3   .  E8 D2E90600 CALL <JMP.&mfc42.#4224>                ;Reg_Fail..!!
004D3BF8   .  68 AA850000 PUSH 85AA
004D3BFD   .  8BCE        MOV ECX,ESI
```

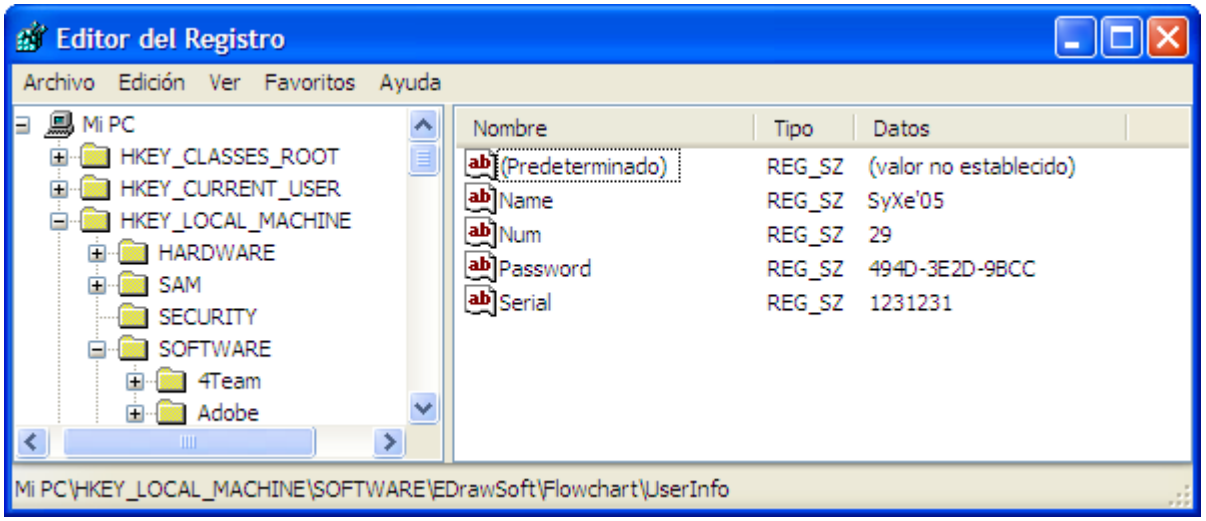
En :4D3BF3 nos saca el mensaje de "serial_inválido" entonces sabemos que no debería haber saltado en :4D3B4E porque así hubiéramos evitado pasar por el CALL que nos saca este mensaje ya que llegamos a un JMP que nos pone directamente en :4D3C49, esto hay que investigarlo: una llamada a "CGdi.dll" de esta depende ver el mensaje de error o no, en función del valor de retorno EAX==0 ó EAX==1, dejamos correr el programa y que nos muestre el mensaje de error, volvemos a intrducir los datos y ahora cuando vaya a saltar en :4D3B4E cambiamos el Flag_Zero para que no salte, pulsamos F9 y…



Vaya el mensaje de felicitación, jeje.. el responsable es el CALL que hay en :4D3B7A, veamos la zona más ampliamente pues se entiende mejor:

```
004D3B47  .   E8 02100700  CALL <JMP.&cgdi.?RegeditSoft@CCrypte;  ;Ojo a este CALL..
004D3B4C  .   85C0         TEST EAX,EAX
004D3B4E  .v  74 79        JE SHORT tute1.004D3BC9
004D3B50  .   8D4C24 0C    LEA ECX,DWORD PTR SS:[ESP+C]
004D3B54  .   E8 D1DE0600  CALL <JMP.&mfc42.#540>
004D3B59  .   68 78C90000  PUSH 0C978
004D3B5E  .   8D4C24 10    LEA ECX,DWORD PTR SS:[ESP+10]
004D3B62  .   C74424 28 02 MOV DWORD PTR SS:[ESP+28],2
004D3B6A  .   E8 F1DE0600  CALL <JMP.&mfc42.#4160>
004D3B6F  .   8B4424 0C    MOV EAX,DWORD PTR SS:[ESP+C]
004D3B73  .   6A 00        PUSH 0
004D3B75  .   6A 00        PUSH 0
004D3B77  .   50           PUSH EAX
004D3B78  .   8BCE         MOV ECX,ESI
004D3B7A  .   E8 4BEA0600  CALL <JMP.&mfc42.#4224>          ;Reg_Succeed..!!
004D3B7F  .   68 54C90000  PUSH 0C954
004D3B84  .   8D4C24 10    LEA ECX,DWORD PTR SS:[ESP+10]
004D3B88  .   E8 D3DE0600  CALL <JMP.&mfc42.#4160>
004D3B8D  .   E8 0ADF0600  CALL <JMP.&mfc42.#1175>
004D3B92  .   85C0         TEST EAX,EAX
004D3B94  .v  74 09        JE SHORT tute1.004D3B9F
004D3B96  .   8B10         MOV EDX,DWORD PTR DS:[EAX]
004D3B98  .   8BC8         MOV ECX,EAX
004D3B9A  .   FF52 7C      CALL DWORD PTR DS:[EDX+7C]
004D3B9D  .v  EB 02        JMP SHORT tute1.004D3BA1
004D3B9F  >   33C0         XOR EAX,EAX
004D3BA1  >   8B4C24 0C    MOV ECX,DWORD PTR SS:[ESP+C]
004D3BA5  .   51           PUSH ECX
004D3BA6  .   8D88 B000000 LEA ECX,DWORD PTR DS:[EAX+B0]
004D3BAC  .   E8 73DE0600  CALL <JMP.&mfc42.#860>
004D3BB1  .   8BCE         MOV ECX,ESI
004D3BB3  .   E8 B8E00600  CALL <JMP.&mfc42.#4853>
004D3BB8  .   C74424 24 FF MOV DWORD PTR SS:[ESP+24],-1
004D3BC0  .   8D4C24 0C    LEA ECX,DWORD PTR SS:[ESP+C]
004D3BC4  .v  E9 80000000  JMP tute1.004D3C49
004D3BC9  >   8D4C24 10    LEA ECX,DWORD PTR SS:[ESP+10]
004D3BCD  .   E8 58DE0600  CALL <JMP.&mfc42.#540>
004D3BD2  .   68 79C90000  PUSH 0C979
004D3BD7  .   8D4C24 14    LEA ECX,DWORD PTR SS:[ESP+14]
004D3BDB  .   C74424 28 03 MOV DWORD PTR SS:[ESP+28],3
004D3BE3  .   E8 78DE0600  CALL <JMP.&mfc42.#4160>
004D3BE8  .   8B5424 10    MOV EDX,DWORD PTR SS:[ESP+10]
004D3BEC  .   6A 00        PUSH 0
004D3BEE  .   6A 00        PUSH 0
004D3BF0  .   52           PUSH EDX
004D3BF1  .   8BCE         MOV ECX,ESI
004D3BF3  .   E8 D2E90600  CALL <JMP.&mfc42.#4224>          ;Reg_Fail..!!
004D3BF8  .   68 AA850000  PUSH 85AA
```

Vemos el CALL que decide si saltará o no en :4D3B4E, si salta vemos el mensaje de error y si no lo hace el de felicitación, de esto deducimos que se trata de la rutina de verificación del serial y se encuentra en "CGdi.dll" que está empakada con ASPack 2.12 también, antes aunque nos haya mostrado el mensaje bueno realmente no ocurrió nada pues seguimos en Modo_Trial, ya os comenté que no me gustó que saltará en :100017A4 evitando el CALL WriteRegeditInfo@Crypter que como su nombre indica se encarga de escribir los datos en el registro de Window$, probé a cambiar el Flag_Zero estando en :100017A4 y que pasara por dicho CALL y efectivamente se escribieron los datos en el registro ;-)



Allí quedaron registrados ;-)

Si ahora miramos el programa no vemos referencias a Trial ni nada de esto, además si miramos en el About encontramos nuestro nombre allí:

O sea que estamos registrados, si reiniciamos el programa seguimos licenciados, esto es gracias a que los datos se escribieron realmente en el registro y ahora va allí y lo encuentra, un parche fácil y sin modificar el .exe ni la .dll sería crearnos un archivo con extensión .reg donde poner esto:

```
-------/ inicio_archivo
Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\SOFTWARE\EDrawSoft\Flowchart\UserInfo]
"Num"="18"
"Name"="SyXe'05"
"Serial"="1132123"
"Password"="484C-3E2D-9ACD"
-------/ final_archivo
```

Este sólo lo debemos de ejecutar en nuestra máquina, entonces escribirá allí los datos por nosotros y programa crackeado ;-))

Espero que os hayáis divertido un rato, de esto se trataba de pasarlo bien y aprender algo si se puede, si no para otro día más, jeje…

Un saludo para todo CracksLatinoS sin excepción alguna xDD

Para cualquier cosa ya sabéis donde estoy:

syxe00@yahoo.es